

Struttura di un Programma C++

Struttura di un Programma C++

Le parti fondamentali che compongono ogni programma C++.

Come è fatto un programma C++?

Prima di scrivere il tuo primo programma, devi capire la sua struttura. È come imparare a costruire una casa: devi sapere dove vanno le fondamenta, i muri e il tetto, prima di poter arredare.

Ogni programma C++ segue una struttura fissa. Conoscerla ti permette di capire qualsiasi codice tu incontrerai.

La struttura minima

Ecco il programma C++ più semplice possibile:

```
#include <iostream>
using namespace std;

int main() {
    cout << "Ciao!";
    return 0;
}
```

Questo programma stampa la parola "Ciao!" sullo schermo. Solo cinque righe, ma ognuna ha un ruolo preciso.

Parte 1: `#include <iostream>`

```
#include <iostream>
```

Questa riga dice al compilatore: "Ho bisogno della libreria `iostream`."

Pensa alle librerie come a delle cassette degli attrezzi: `iostream` è la cassetta che contiene gli strumenti per scrivere sullo schermo (`cout`) e leggere l'input dell'utente (`cin`).

Senza questa riga, il compilatore non saprebbe cosa significa `cout` e il programma non funzionerebbe.

Parte 2: `using namespace std;`

```
using namespace std;
```

Pensa a un **namespace** come a un cognome. Se in classe ci sono due ragazzi chiamati Marco, li distingui con il cognome: "Marco Rossi" e "Marco Bianchi".

`cout` e `cin` appartengono alla "famiglia" `std`. Scrivere `using namespace std;` dice al programma: "quando uso `cout`, intendo `std::cout`". Così puoi scrivere `cout` invece del nome completo `std::cout`.

Parte 3: `int main() { ... }`

```
int main() {  
    // il programma inizia qui  
    return 0;  
}
```

La funzione `main()` è il **punto di partenza** di ogni programma C++. Quando avvii un programma, il computer cerca sempre `main()` e inizia a eseguire le istruzioni al suo interno.

- `int` — indica che `main` restituisce un numero intero al termine
- Le parentesi graffe `{` e `}` — delimitano il corpo della funzione
- `return 0;` — segnala che il programma è terminato senza errori (0 = tutto ok)

Parte 4: le istruzioni dentro `main()`

```
cout << "Ciao!";
```

Dentro `main()` scrivi le istruzioni del programma, una per riga. Ogni istruzione termina con il punto e virgola `;`.

`cout` stampa qualcosa sullo schermo. La freccia `<<` indica "manda questo testo a cout".

Un esempio più completo

Quando hai bisogno di più funzionalità, includi più librerie:

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    // Dichiariamo due variabili
    string nome = "Alice";
    int eta = 16;

    // Le stampiamo sullo schermo
    cout << "Nome: " << nome << endl;
    cout << "Età: " << eta << endl;

    return 0;
}
```

Output:

```
Nome: Alice
Età: 16
```

Qui abbiamo aggiunto `#include <string>` perché vogliamo usare le stringhe di testo.

L'ordine delle sezioni

Un file C++ tipico segue sempre quest'ordine dall'alto verso il basso:

1. **Le librerie** — cosa includiamo
2. **Il namespace** — quale "famiglia" di nomi usare
3. **Variabili e costanti globali** — valori disponibili ovunque (opzionale)
4. **La funzione `main()`** — da dove parte il programma
5. **Altre funzioni** — pezzi di codice riutilizzabili (opzionale)

```
// 1. Librerie
#include <iostream>
#include <cmath>

// 2. Namespace
using namespace std;

// 3. Costante globale (opzionale)
const double PI = 3.14159;

// 4. Funzione main
int main() {
    double raggio = 5.0;
    // PI * r * r calcola l'area del cerchio
    double area = PI * raggio * raggio;
    cout << "Area: " << area << endl;
    return 0;
}
```

Le parentesi graffe delimitano i blocchi

Le parentesi graffe `{ }` compaiono ovunque in C++: nelle funzioni, nei cicli, nelle condizioni. Ogni coppia apre e chiude un **blocco di codice**.

```
int main() {           // apre il blocco di main
    if (true) {        // apre un blocco if
        cout << "Ciao";
    }                  // chiude il blocco if
    return 0;
}                      // chiude il blocco di main
```

Ogni `{` deve avere il suo `}` corrispondente. Se ne manca uno, il compilatore segnala un errore.

Il punto e virgola alla fine di ogni istruzione

In C++, quasi tutte le istruzioni terminano con `;`. Dimenticarlo è uno degli errori più comuni:

```
// Corretto
cout << "Ciao!" << endl;
int x = 5;

// Errato – manca il ;
cout << "Ciao!" << endl // il compilatore segnala un errore
int x = 5               // anche qui
```

Se non capisci un errore del compilatore, prima di tutto controlla che non manchi un punto e virgola.